

Migración de Monolito a Microservicios

Integrantes: Joaquín Fuenzalida, Benjamín Vallejos

Profesor: Matías Vargas

Fecha: 30 de marzo de 2026

Tabla de contenidos

1. Introducción.....	3
2. Descripción del sistema.....	3
2. 1. Contexto general.....	3
2. 2. Alcance.....	4
3. Análisis arquitectónico AS-IS.....	5
3. 1. Descripción de la arquitectura actual.....	5
3. 2. Requerimientos.....	5
3. 2. 1. Requerimientos funcionales.....	5
3. 2. 2. Requerimientos no funcionales.....	5
3. 3. Problemas identificados.....	5
3. 4. Diagramas AS-IS.....	6
4. Diseño arquitectónico TO-BE.....	7
4. 1. Estrategia de migración.....	7
4. 2. Definición de microservicios.....	7
4. 3. Arquitectura general.....	8
4. 4. Comunicación entre servicios.....	8
4. 5. Diagrama de secuencia.....	9
5. Patrones arquitectónicos.....	10
5. 1. API Gateway.....	10
5. 2. Saga Pattern.....	10
5. 3. Circuit Breaker.....	10
5. 4. Strangler Pattern.....	11
6. Implementación técnica.....	12
6. 1. Arquitectura de despliegue.....	12
6. 2 Docker compose.....	12
6. 3. Base de datos.....	13
7. Observabilidad.....	14
7. 1. Herramientas utilizadas.....	14
7. 2. Métricas.....	15
7. 3. Dashboards.....	17
7. 4. Análisis de métricas.....	19
7. 5. Estimación de costos.....	20
7. 6. Análisis.....	20
8. Trade-offs.....	22
9. Conclusiones.....	23
10. Referencias.....	24

1. Introducción

El objetivo del proyecto consiste en realizar la migración de un sistema que contiene arquitectura monolítica a una basada en microservicios, buscando evaluar su estado actual, diseñando una mejora del mismo y ejecutando un prototipo funcional demostrando la operabilidad entre servicios e implementando estrategias de observabilidad.

A nivel técnico, la transición de monolito a microservicios contempla la separación del dominio en servicios independientes, cada uno con responsabilidades claras y límites funcionales definidos, permitiendo una evolución mucho más ordenada del sistema. Además, se incorpora un apartado de observabilidad, con el fin de medir el comportamiento en tiempo real del sistema durante su ejecución, monitoreando métricas de latencia, throughput, tasa de errores y uso de recursos. Esto nos permite realizar la comparación del desempeño entre el sistema de arquitectura monolítica a microservicios, fundamentando las decisiones de la arquitectura en base a su evidencia

El propósito de la migración de una arquitectura monolítica a microservicios no es “dividir” el sistema, sino mejorar su mantenibilidad, resiliencia y capacidad de crecimiento. En este sentido, el proyecto busca demostrar que una arquitectura de microservicios bien estructurada puede lograr facilitar la escalabilidad por cada servicio independiente, reduciendo el impacto de cambios sobre el sistema completo y habilitando una operación más controlada mediante el monitoreo continuo de uso de recursos y tiempos de respuesta por operación.

2. Descripción del sistema

2. 1. Contexto general

El siguiente sistema trata sobre una aplicación de tracking de paquetes, la cual gestiona el seguimiento en tiempo real de envíos y entregas de paquetes, la cual posee una arquitectura de software mal diseñada.

El sistema opera con entidades clave, tales como usuarios, paquetes y tracking de paquetes, gestionando estados de los paquetes (created, in_transit, out_for_delivery, delivered y exception). Estos estados representan hitos de negocio relevantes para la operación logística y para la experiencia del usuario final, el que requiere de información confiable respecto al estado de envío de su producto.

En su estado inicial, el sistema posee una arquitectura monolítica con un mal diseño, lo que genera acoplamiento entre componentes, dificultades para lograr mantener la calidad del servicio bajo carga, dificultades para mantener la calidad del servicio bajo carga y limitaciones para incorporar nuevas funcionalidades sin afectar otras partes del sistema.

Bajo este escenario, la migración a una arquitectura de microservicios se plantea como una estrategia para lograr desacoplar dominios, distribuir responsabilidades y mejorar la capacidad de respuesta técnica y operativa del sistema.

2. 2. Alcance

El sistema cuenta con apartados como:

- Gestión de usuarios: El sistema posee un método POST (/createUser) y un GET (/getUsers), donde el primer método consta de la creación de un usuario solicitando un nombre y correo electrónico, mientras que el otro trata de la muestra de todos los usuarios registrados en el sistema.
- Gestión de paquetes: La aplicación posee dos endpoints para la administración de paquetes, donde el primero trata de un método POST (/createPackage), donde requiere el registro de los siguientes datos: user_id, package_title (o descripción del paquete), city (la ciudad de origen) y location (la ciudad de destino). Al realizar este método, se crea un código para su seguimiento futuro. Como tal, el sistema provee a su vez un método GET (getAllPackages), el cual devuelve una lista con el estado actual de cada paquete registrado.
- Sistema de seguimiento de paquetes: Dentro de esta categoría, se encuentra un método POST (/updateStatus) el cual actualiza el estado de envío de un paquete, solicitando al usuario colocar: tracking_code (el código de seguimiento de paquete generado) y new_status (nuevo estado del envío), dando las opciones CREATED, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERED y EXCEPTION. Por otra parte, se encuentra un GET (/getTracking), el cual solicita tracking_code para consultar el estado del envío del paquete.
- Métricas: En este apartado, se muestran valores capturados de la base de datos mediante un método GET, otorgando la cantidad de paquetes distintos en el sistema, el total de eventos, tiempo promedio de proceso (de manera simulada) y la hora de la solicitud.

Lo que no incluye el sistema es lo siguiente:

- Sistema de usuario: La aplicación no cuenta con una autenticación para la verificación de identidad del usuario para el acceso seguro a la aplicación, además no se cuenta con los roles de usuario para el ingreso a ciertas funcionalidades del sistema.
- Notificaciones de estado de paquete: Se solicita un correo electrónico al usuario, sin embargo no se le da un uso a este como lo puede ser el envío de correos electrónicos sobre el estado del paquete.
- Confirmación de entrega: No cuenta con algo que pueda comprobar la entrega exitosa del paquete.
- Puntos de distribución: No se tiene en consideración los diversos puntos de distribución de paquetes
- Facturación: No se tiene un sistema de procesamiento de pagos, ni de cálculo de costos de envío
- Canal de ayuda: No cuenta con un canal de ayuda en caso de no saber requerimientos de envío, la administración del envío, devoluciones de paquetes, reembolso o números de contacto.
- Panel de administración: No se tiene un panel donde un administrador pueda gestionar paquetes o visualizar dashboard de estos.

3. Análisis arquitectónico AS-IS

3. 1. Descripción de la arquitectura actual

El backend se concentra en un único módulo HTTP, dando a conocer las reglas del negocio, acceso a sus datos y eventos de notificación mediante la consola.

El frontend consume endpoints directamente desde el backend.

La persistencia de sus datos utiliza una tabla central que combina la identidad del usuario, paquete y eventos de estado de paquetes (envío).

3. 2. Requerimientos

3. 2. 1. Requerimientos funcionales

- **RF-1:** El sistema permite crear usuarios con username y email.
- **RF-2:** El sistema permite crear paquetes asociados a un usuario con tracking único.
- **RF-3:** El sistema permite actualizar el estado de seguimiento de un paquete.
- **RF-4:** El sistema permite ver seguimiento de un paquete.

3. 2. 2. Requerimientos no funcionales

- **RNF-1:** Disponibilidad local de los servicios
- **RNF-2:** Baja latencia de consulta para el tracking
- **RNF-3:** Persistencia de transacciones con PostgreSQL
- **RNF-4:** Despliegue del sistema mediante docker

3. 3. Problemas identificados

- **Deuda técnica:** Endpoints de tipo NO REST (genera dificultades en el manejo de errores, de difícil mantenimiento, no distingue la causa técnica del error) y este responde con mensajes de error ambiguos “something went wrong”, “no ok”, “fallo red”; no se logra diferenciar entre errores de validación, reglas del negocio, integridad de datos o fallas de infraestructura; existe poco contexto al momento de ocurrir problemas, lo que dificulta la identificación del origen del fallo.
- **Acoplamiento crítico:**
El sistema presenta acoplamiento crítico en lo siguiente:
 - La lógica del negocio, transporte y persistencia de datos, se encuentran mezclados dentro de una sola unidad ([/main.py](#)).
 - Frontend acoplado a una URL absoluta desde el backend.
 - Notificaciones simuladas acopladas al backend principal, ya que al ejecutarse dentro del mismo flujo de petición, una futura falla de notificación puede degradar operaciones fundamentales (tales como crear/actualizar tracking).
 - Los estados del negocio están hardcodeados en el mismo módulo de la API, dificultando el versionamiento y evolución de las reglas
 - Duplicación de datos de dominio en tracking_data, ya que este replica el “username” y “email”, rompiendo el principio “SSOT” (single source of truth).

- **Cuellos de botella:**

- Query's de /getUsers y /getAllPackages: Si se realiza una query de estos endpoints, el sistema entrega absolutamente todos los usuarios y paquetes registrados, la cual puede saturar el sistema si es que existen demasiados usuarios/paquetes.
- Base de datos única: El sistema posee la información del usuario y del paquete mezclados en una sola base de datos, la cual puede generar errores si ocurre un error en la base de datos, se pierde el acceso a toda la información del sistema.
- Posibles datos duplicados en base de datos: Al crear un paquete, el sistema genera automáticamente un código para su seguimiento (tracking_code), donde incluye un prefijo (TRK), un número aleatorio entre 100.000 y 999.999 y una parte basada en un timestamp, tomando únicamente sus últimos 4 dígitos. Lo que finalmente deja un código basado en la aleatoriedad, pero con posibilidad de que se repita en la base de datos, afectando en su unicidad.
- Falta de Primary y Foreign Key: Al no tener una llave única, pueden crearse filas repetidas, la actualización y borrado de datos puede complicarse, y la llave foránea afecta en la llamada de claves primarias dentro de otras tablas, generando datos duplicados por cada query o ingreso de información

3. 4. Diagramas AS-IS

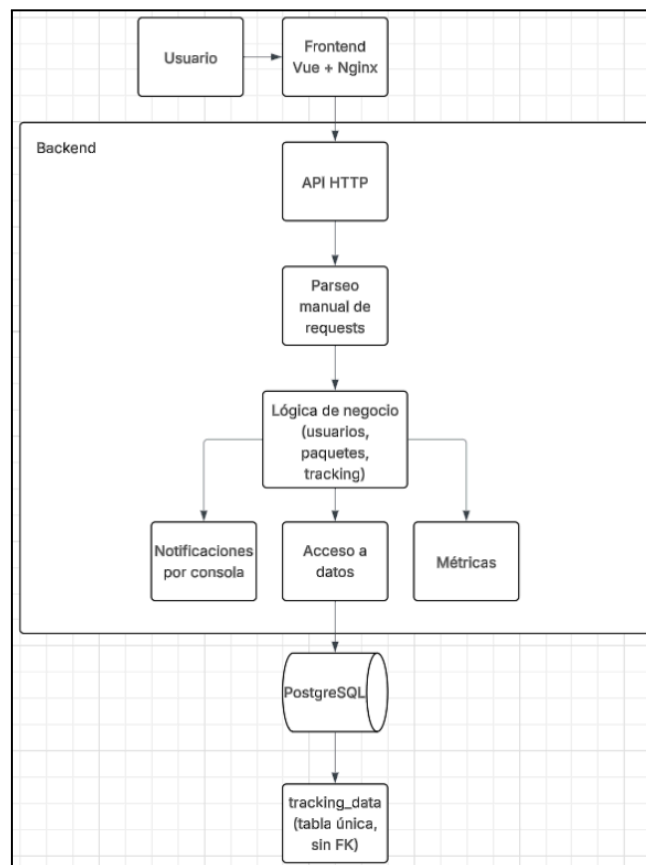


Figura 1. Diagrama de arquitectura monolítica

4. Diseño arquitectónico TO-BE

4. 1. Estrategia de migración

Dentro del diseño arquitectónico que se quiere proponer, realizaremos la migración a el patrón de diseño “Strangler Pattern”, ya que este mejoraría el sistema actual sin la necesidad desde cero toda su estructura, ya que mantener su arquitectura monolítica mantendría los problemas ya existentes dentro del sistema. Esto mantendría su continuidad operativa, ya que se implementarán mejoras de manera gradual.

El API Gateway permitirá el enrutamiento del tráfico entre los componentes legacy y los nuevos servicios a implementar, habilitando su “convivencia temporal” sin interrupción.

La salida de cada fase está condicionada a los siguientes criterios: estabilidad funcional, pruebas de regresión aprobadas, métricas de latencia/error dentro de los umbrales y trazabilidad operativa.

La migración se realizará de la siguiente forma:

- Introducir API gateway delante del monolito
- Extraer el servicio de usuarios
- Extraer el servicio de paquetes
- Extraer servicio de Tracking/Eventos
- Aislar notificaciones asíncronas
- Quitar endpoints legacy del monolito

Se dará término a la migración cuando los endpoints legacy estén totalmente retirados, sus dominios desacoplados y con observabilidad total del sistema.

4. 2. Definición de microservicios

- Servicio 1: Servicio de usuario (registro, login y permisos de usuario)
- Servicio 2: Servicio de paquetes (creación de paquetes, obtención de estado actual del paquete)
- Servicio 3: Servicio de seguimiento (actualización de estado de envío, historial del paquete)
- Servicio 4: Servicio de notificaciones (Aviso estado paquete)

4. 3. Arquitectura general

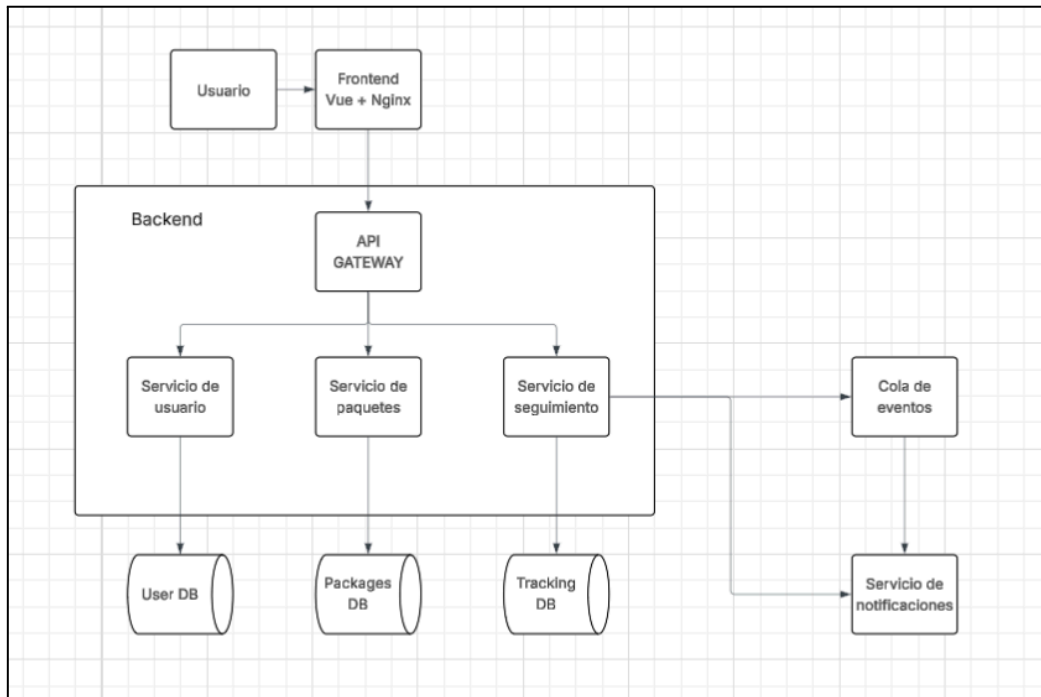


Figura 2. Diagrama de arquitectura general del sistema

4. 4. Comunicación entre servicios

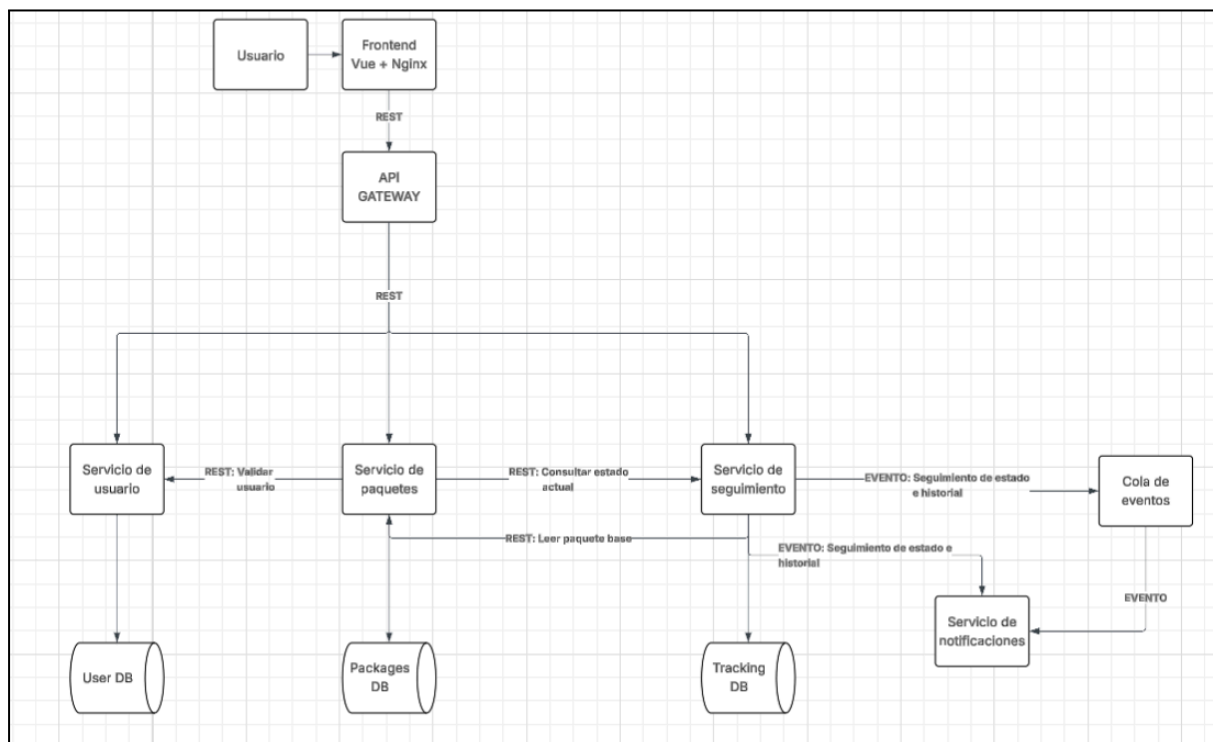


Figura 3. Diagrama de comunicación entre servicios

4. 5. Diagrama de secuencia

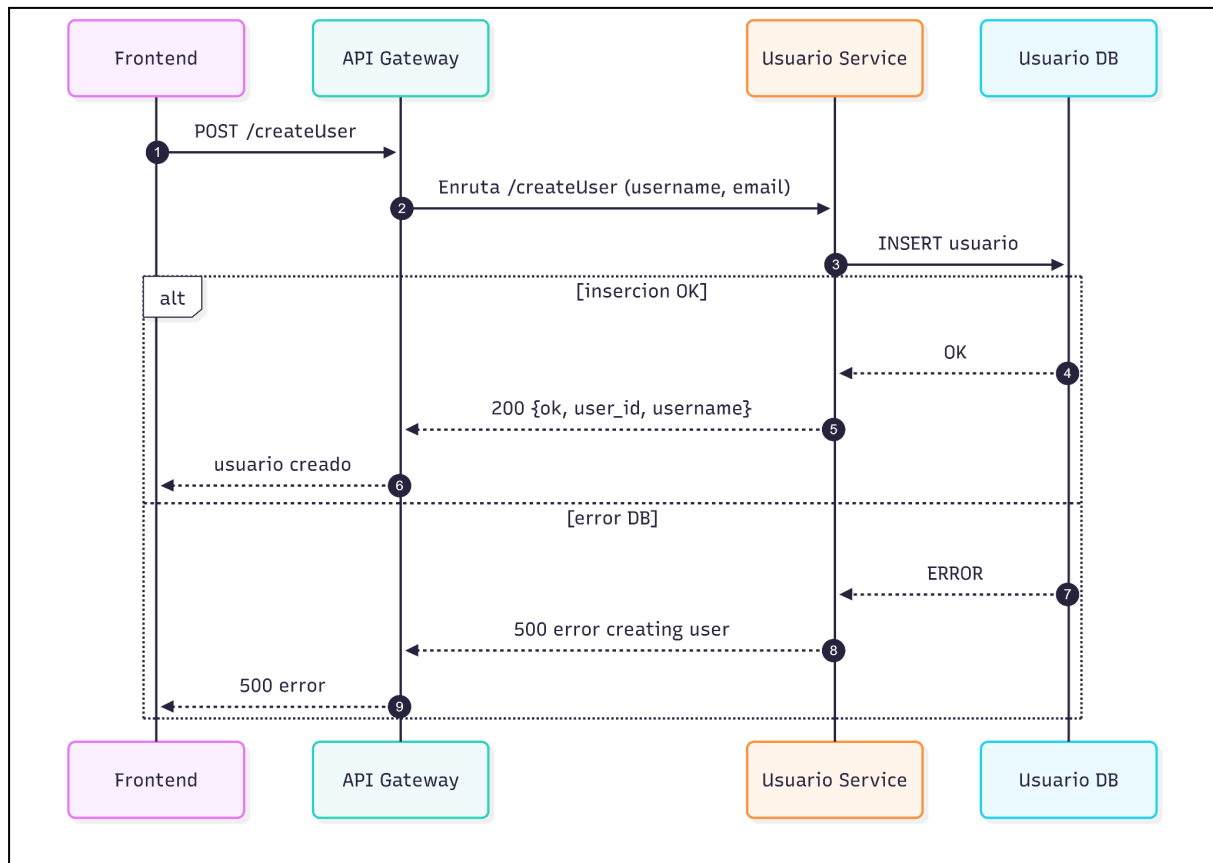


Figura 4. Diagrama de secuencia servicio de usuario

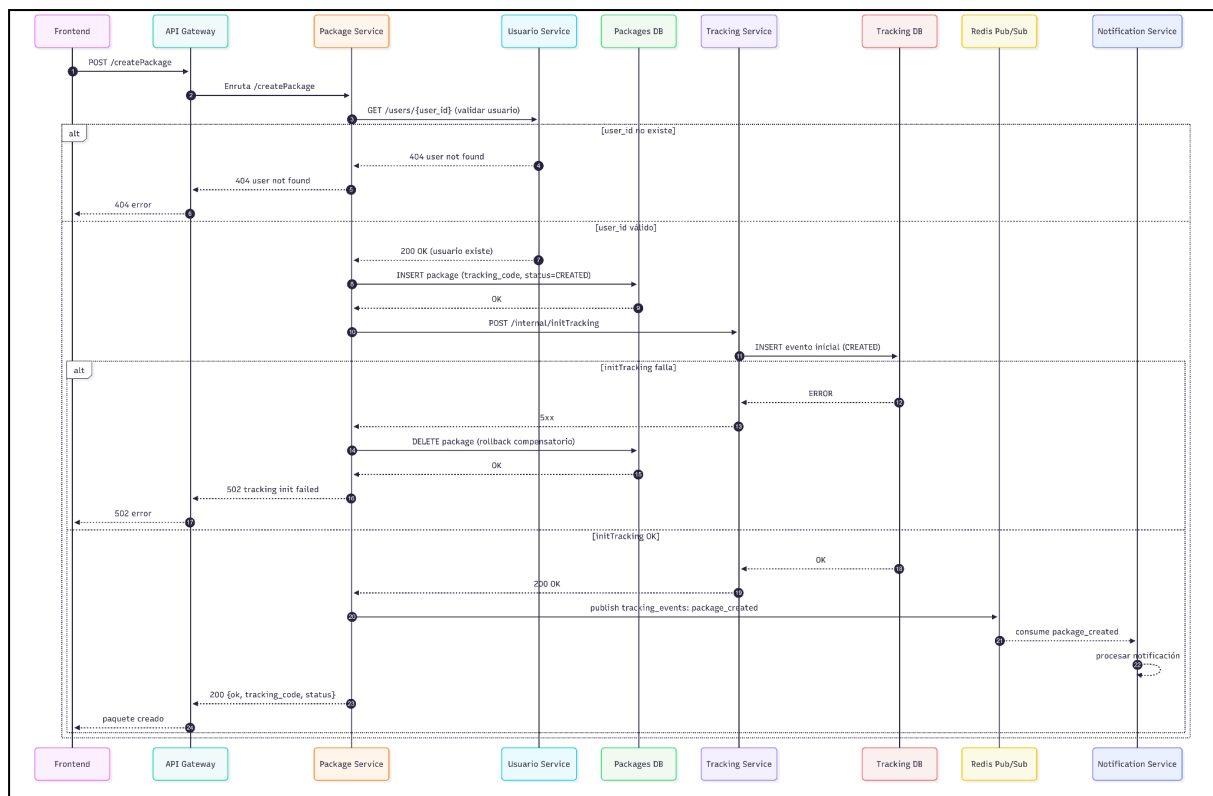


Figura 5. Diagrama de secuencia de servicio de paquetes

5. Patrones arquitectónicos

5. 1. API Gateway

Problema: Múltiples servicios exponen API's y hacen complejo el consumo del frontend.

Solución: Un gateway tiene la capacidad de centralizar rutas, CORS y políticas transversales.

Justificación:

- Reduce el acoplamiento del módulo cliente y permite la evolución interna del sistema sin romper contratos públicos.
- Permite la encapsular la deuda técnica del monolito detrás de una interfaz más estable, reduciendo su impacto en el frontend durante la migración
- Facilita despliegues incrementales, ya que una ruta puede migrar a microservicios, mientras otras siguen apuntando a la arquitectura monolítica

5. 2. Saga Pattern

Problema: Operaciones distribuidas (creación de paquete -> registro de primer estado -> notificación) pueden fallar de manera parcial.

Solución: Saga Choreography mediante eventos con acciones compensatorias (ejemplo: anular paquete).

Justificación:

- Modela correctamente la realidad operativa del negocio:
 - crear paquete
 - registrar estado inicial
 - notificar
- Permite compensaciones ante fallos parciales (ejemplo: mal envío en estado de excepción o revertir algún registro incompleto). Evita los bloqueos prolongados por esperas entre servicios, mejorando su robustez bajo carga y latencias variables.

5. 3. Circuit Breaker

Problema: Falla de un servicio dependiente puede propagar la latencia y timeout a todo el sistema

Solución: Circuit breaker en llamadas del gateway a servicios y entre servicios críticos

Justificación:

- Mejora la resiliencia y limita el efecto de cascada.
- Reduce la saturación de la infraestructura al cortar de manera temporal las llamadas a servicios en falla, evitando saturar con reintentos.
- Entrega señales operativas claras, útiles para su observabilidad y respuesta ante posibles fallos.

5. 4. Strangler Pattern

Problema: La reescritura completa del monolito es riesgosa y de altos costos

Solución: Extracción de manera incremental las capacidades del negocio, manteniendo su continuidad operativa sin interrumpir su servicio.

Justificación:

- Reduce el riesgo de entrega y habilita su validación mediante iteraciones.
- Es coherente con el estado actual del sistema, manteniendo una base funcional que no conviene desechar mediante la reescritura total del sistema.
- Minimiza el riesgo de la interrupción operativa del sistema, permitiendo que sus rutas legacy y rutas migradas coexistan.
- Habilita el rollback por dominio, ya que si una extracción falla, se puede redirigir el tráfico al monolito, sin comprometer el sistema completo.

6. Implementación técnica

6. 1. Arquitectura de despliegue

Primeramente, se realizará un desglose de contenedores por capa, donde los componentes del sistema deberán vivir en un contenedor para respetar el principio de responsabilidad única.

Frontend con Vue.js + nginx:

Interfaz web que consume mediante HTTP los endpoints del backend para el flujo de usuarios, paquetes y tracking. Con nginx actuando como un servidor web ligero.

API Gateway:

Un contenedor dedicado (usando Kong/Traefik) recibiendo el tráfico de internet y redireccionando internamente.

Contenedores de servicios:

Las funcionalidades de usuario, packages y tracking, definidos como contenedores independientes

Persistencia de datos en PostgreSQL para los usuarios, tracking y los paquetes

Cada servicio posee una base de datos única para evitar el acoplamiento de datos, logrando que los servicios sean autónomos, escalables e independientes

Contenedor de cola de eventos

Un contenedor para el sistema broker de mensajería utilizando Redis, teniendo un volumen persistente, para que en caso de que el contenedor falle, los mensajes de la Saga Pattern no se pierdan

6. 2 Docker compose

Se implementa como mecanismo de orquestación para poder levantar el entorno completo sin dificultad alguna.

El archivo contiene los siguientes servicios:

- Backend, la que contiene la lógica de usuarios, paquetes y tracking
- Frontend, que publica la interfaz de Vue mediante Nginx
- Contenedores de bases de datos, cada servicio tiene su propia instancia de base de datos (auth, packages, seguimiento)

La composición de este archivo entrega el nivel de aislamiento de servicios necesario para pruebas de integración, sin exigir una plataforma de orquestación más compleja.

6. 3. Base de datos

Se utiliza PostgreSQL como persistencia central del sistema, debido a su estabilidad transaccional y adopción amplia.

Se utiliza para separar cada información de cada servicio, siendo independiente cada uno, tales como la creación de usuarios, registro de paquetes, actualización de estados e historial de tracking por id. *

El modelo de tracking implementa un enfoque append-only, lo que significa que cada cambio de estado se registra como un nuevo evento en vez de sobrescribir el estado previo. Esta decisión permite mantener la trazabilidad histórica completa del sistema.

7. Observabilidad

7. 1. Herramientas utilizadas

Para garantizar la continuidad operativa del sistema, es necesario aplicar herramientas de monitoreo como lo son Prometheus y Grafana, los cuales suelen ser el estándar de la industria para los entornos de contenedores.

Prometheus:

Es un sistema de monitorización y alertas de código abierto, el cual recolecta métricas en tiempo real de cada contenedor mediante los endpoints de cada uno, almacenando la información recolectada con marcas de tiempo. Además, envía una notificación si detecta alguna anomalía dentro de los componentes del servidor (ej. CPU +80% para un servicio).

Grafana:

Es una plataforma de código abierto para la visualización y análisis de datos en tiempo real lo que ayuda a la toma de decisiones para el sistema. Lo que hace Grafana, consiste en la muestra de dashboards interactivos que muestran la salud global del sistema, por otra parte, permite la observación de correlaciones entre componentes del sistema (ej. un fallo en Servicio de seguimiento afecta la latencia percibida en la API Gateway).

El uso de estas dos herramientas nos ayudan a validar los patrones implementados en el sistema, tales como:

- Monitoreo del Saga Pattern: Se medirán métricas específicas de la cola de eventos como el número de “acciones compensatorias” ejecutadas en el sistema (si hay muchas compensaciones significa que hay un fallo en la consistencia del negocio).
- Validación del Circuit Breaker: Prometheus registrará cada vez que el Circuit Breaker cambie al estado “Abierto”, permitiendo que Grafana muestre alertas visuales de manera inmediata en caso de un fallo en un servicio.
- Control del Strangler Pattern: Se usarán dashboards comparativos en Grafana para asegurar que el nuevo microservicio tenga un rendimiento superior al sistema legacy antes de cerrarlo por completo.
- Métricas RED (Rate-Errors-Duration): Se implementará el monitoreo de la tasa de peticiones, errores y duración/latencia en cada punto del API Gateway para garantizar que el usuario tenga una experiencia fluida en la aplicación.

7. 2. Métricas

Para realizar la comparación entre la arquitectura monolítica y la arquitectura de microservicios, se realizarán métodos POST de manera que se pueda medir:

- Latencia
- Throughput
- Tasa de errores
- Uso de recursos

Para esto, se usarán las siguientes peticiones mediante un script con los siguientes parámetros de estrés para ambos tipos de arquitectura (monolítica y microservicios):

- workers = 4 (Número de generadores en paralelo)
- users = 120 (Cantidad de usuarios creados por worker)
- packages = 25 (Creación de paquetes por usuario creados por cada worker)
- reads = 1500 (Ciclo de lectura/actualización mixtos por worker, ejecutando varias operaciones como getUsers, getAllPackages, updateStatus y getTracking)
- delay = 2 ms (Espera entre iteraciones)

Para saber el total de consultas, se tomarán los datos propuestos de manera que queda así:

$$Worker = 120\ users + (120 * 25)\ packages + (1500 * 4)\ reads$$

$$Worker = 120 + 3000 + 6000$$

$$Worker = 9120$$

Por cada worker, se realizan 9120 consultas, por lo que, al ser 4 de estos, la totalidad de consultas realizadas dan el resultado de 36.480 consultas para el sistema.

Sistema de arquitectura monolítica

VALOR	LATENCIA	THROUGHPUT	TASA DE ERRORES	USO DE RECURSOS
Principio consulta (Imagen 1)	95.7 ms	5.01 req/s	0%	4.6% CPU 80 MB
Posterior a consulta (Imagen 2)	96.8 ms	41 req/s	0%	48% CPU 132 MB
Fin consulta (Imagen 3)	1 s	6.16 req/s	0%	98.1% CPU 152 MB

Tabla 1. Métricas de sistema de arquitectura monolítica

Al ver que el sistema se ve sobrepasado por la cantidad de peticiones y una constante entre los valores de latencia, throughput y uso de recursos, se decidió dar de baja las consultas debido al alto consumo y lentitud de respuesta.

Sistema de arquitectura de microservicios

VALOR	LATENCIA	THROUGHPUT	TASA DE ERRORES	USO DE RECURSOS
Principio consulta	9.50 ms usuario 180 ms package 48.8 ms tracking	0.566 req/s	0%	0.648 cores 560 MB
Posterior a consulta	51.2 ms usuario 835 ms package 690 ms tracking	33.9 req/s	0%	1.20 cores 883 MB
Fin consulta (término del script)	13.7 ms usuario 742 ms package ms tracking	0.0517 req/s	0%	0.0711 cores 921 MB

Tabla 2. Métricas de sistema de arquitectura de microservicios

7. 3. Dashboards

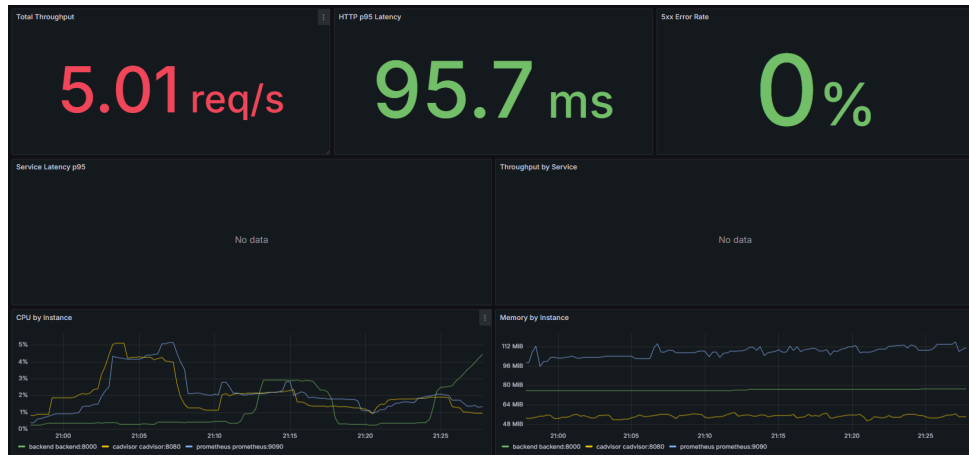


Imagen 1. Dashboard de métricas de arquitectura monolítica.

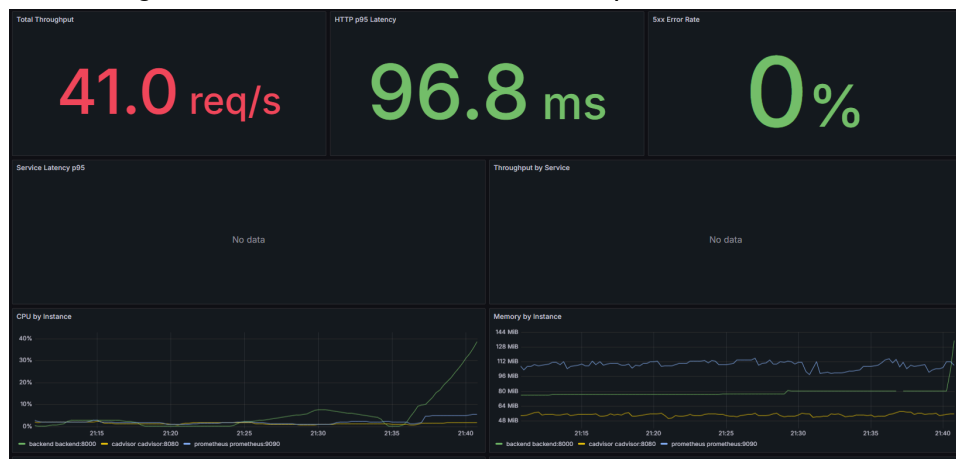


Imagen 2. Dashboard de métricas de arquitectura monolítica.

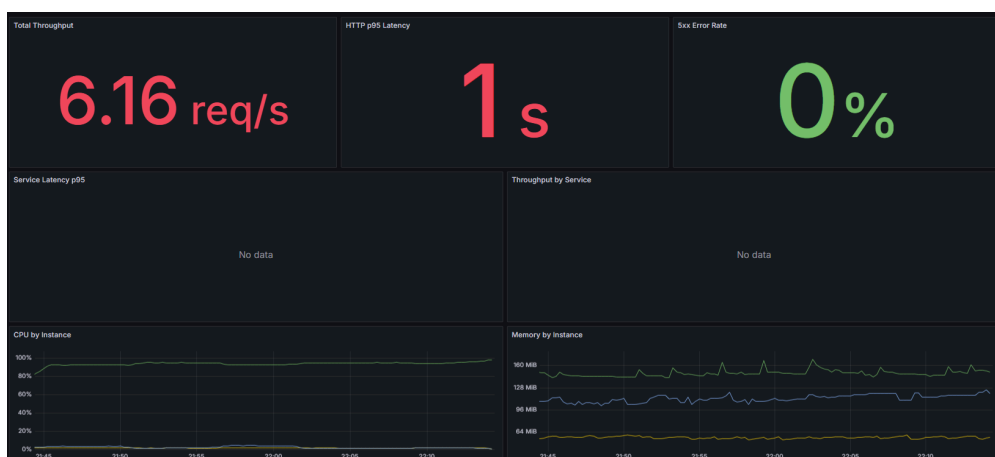
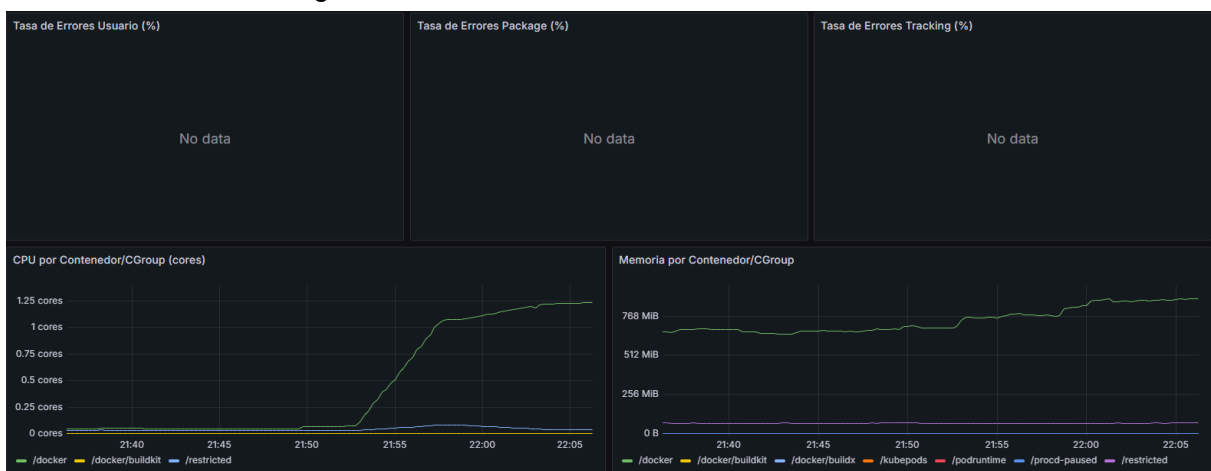


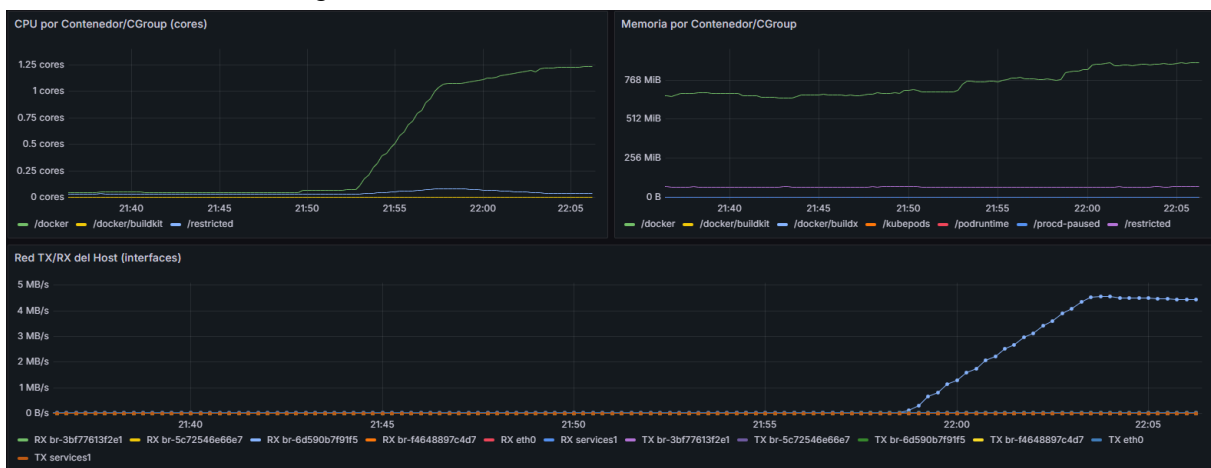
Imagen 3. Dashboard de métricas de arquitectura monolítica.



Imágen 4. Dashboard de métricas de microservicios



Imágen 5. Dashboard de métricas de microservicios



Imágen 6. Dashboard de métricas de microservicios

7. 4. Análisis de métricas

Interpretación de datos del sistema con arquitectura monolítica:

Como tal, se puede apreciar que desde el principio se ve un sistema sano, el cual tenía una latencia de 95.7 milisegundos y un 4.6% de uso de CPU, pero a medida que las consultas iban en aumento, el sistema se fue saturando, generando alzas en la latencia y en el uso de recursos, llegando a puntos críticos como 1 segundo de latencia y un 98% de CPU.

Por otra parte, se puede apreciar el colapso del throughput, donde en el intermedio de la consulta el sistema procesaba 41 req/s con un esfuerzo del 48% de CPU. Sin embargo, en el estado final, el CPU alcanzó casi el punto máximo afectando drásticamente al throughput, viéndose un declive a 6.16 req/s. Esto se debe a la sobrecarga de gestión, donde el servidor gasta energía intentando manejar la cola de espera.

A su vez, es posible visualizar un aumento en el crecimiento de la memoria subiendo de 80 MB a 152 MB, lo que puede ser fatal para una arquitectura monolítica, ya que si una función consume mucha memoria, puede llegar a provocar un error de “Out of memory”, provocando que todo el sistema se caiga.

La tasa de errores se mantuvo en un 0%, ya que el sistema no alcanzó a “caerse”, no obstante, este se encontraba “ahogándose” debido al consumo de la CPU y la latencia. Pero si un usuario, llega a interactuar con el sistema y el tiempo de respuesta es lento, este sentirá que la aplicación está rota aunque no se despliegue un mensaje de error.

Interpretación de datos del sistema con arquitectura de microservicios:

En la arquitectura monolítica se logra apreciar un comportamiento inicialmente estable y sano, con una latencia de 9.50 ms en el servicio de usuarios, 180 ms en el servicio de package y 48.8 ms en el servicio de tracking, junto con un uso de 0.648 cores de CPU y 560MB de memoria.

A medida que las consultas van aumentando, el sistema incrementa su nivel de trabajo y en la etapa posterior, la latencia sube a 51.2 en el servicio de usuarios, 835 ms en el servicio de package y 690 ms en tracking, con un consumo de 1.20 cores de CPU y 883 MB en memoria. Esto refleja que el sistema sigue operativo, pero con presión de por medio en los servicios más acoplados al flujo del negocio, especialmente en package y tracking.

Por otra parte, se puede apreciar el comportamiento del throughput, donde en el intermedio de la consulta, el sistema logra alcanzar las 33.9 req/s, mostrando su mayor capacidad de procesamiento bajo carga. Sin embargo, en el estado final, al término de la ejecución del script, el throughput desciende a 0.0517 req/q, lo que genera coherencia con la caída de la carga activa ya finalizando la prueba. En este punto, el consumo de la CPU disminuye a 0.0711 cores, indicando que ya no existe demanda sostenida, aunque la latencia de package se mantiene alta en 742 ms, señal de que ese servicio en particular sigue siendo el punto más sensible del flujo.

A su vez, se logra visualizar el crecimiento del consumo de la memoria desde 560 MB hasta los 921 MB durante toda la ejecución. Este patrón se mantiene consistente con una arquitectura distribuida (microservicios), en donde los contenedores y procesos mantienen su estado, caché y recursos asignados, incluso cuando la carga disminuye. Esto no implica

una caída del sistema, sino una evidencia del costo base y post-carga en microservicios es mucho mayor, por lo que se vuelve clave el mantener un monitoreo constante de los límites y políticas de liberación de memoria.

7. 5. Estimación de costos

Concepto	Monolito	Microservicios
Infraestructura	RAM: 152 MB CPU: 98% Más barato, menos rendimiento	RAM: 921 MB Core: 1.20 Mayor costo (por cada microservicio), mejor rendimiento
Escalabilidad	Al llegar a 98% CPU, throughput baja de 41 req/s a 6 req/s. Para poder escalar el monolito, debería ser duplicado	El servicio package llegó a tener una latencia de 835 ms. Puede ser escalable mejorando solo este contenedor
Operación	Con un chequeo básico de salud se puede saber el estado del monolito. Su despliegue es demoroso debido a su compilación, además un pequeño error puede llegar a romper todo el sistema.	Es necesaria la implementación de monitoreo por Prometheus y Grafana para saber que ocurre en cada servicio. El despliegue es más rápido debido a su fragmentación.

Tabla 3. Comparativa entre sistemas para estimación de costos.

7. 6. Análisis

Según la tabla, se puede apreciar los distintos tipos de pros y contras respecto a la arquitectura monolítica como la de microservicios:

Infraestructura:

Como se puede visualizar, la arquitectura monolítica es más barata debido a la forma en la que se encuentra estructurada, donde la comunicación entre módulos tiene un costo de \$0, pero como se puede observar, es bastante limitada en cuanto a su consumo. Por otra parte, los microservicios cada vez que el API Gateway se comunica con los servicios, se genera un tráfico de red interno. Por ejemplo, en nubes como AWS si los servicios se encuentran en distintas zonas de disponibilidad para mayor seguridad, el tráfico de red entre estos es cobrado por GB.

Además, los microservicios requieren de un cluster, lo que implica un costo base por el plano de control, el cual suele rondar entre los \$70 y \$100 USD solo por su existencia, antes de la suma de los servidores reales.

Escalabilidad:

Este tópico es de gran importancia debido al alcance de los proyectos, el monolito es ideal para proyectos pequeños o startups debido a su simplicidad, desarrollo y despliegue, mientras que los microservicios son mejores para proyectos de mayor complejidad y con un crecimiento esperado. Por lo cual, si queremos escalar el monolito de este proyecto, es necesario la replicación de todos los módulos del sistema, en este caso, el límite del monolito llegó a 152 MB ocupando solo un Core, si quisiera escalar 10 veces esto, se necesitarán 10 Cores y 1.5 GB.

Por la parte de los microservicios, la escalabilidad es más fácil debido a su granularidad, donde solamente debe gastarse dinero en un módulo en particular (ej. packages que tuvo una latencia de 835 ms).

Operación:

Para poder saber el estado del sistema, en el monolito basta con realizar un chequeo básico del sistema completo, comprobando métricas como la latencia, throughput o los usos de recursos, mientras que por la parte de microservicios es un poco más complicado y demoroso, debido a los diferentes servicios que se ofrecen, teniendo que analizar cada uno de ellos y probándolos para visualizar posibles fallos.

Si de fallos hablamos, es necesario saber el MTTR (Mean Time To Recovery) de estas dos arquitecturas, si el monolito llega a fallar en algo, como lo fue en la CPU al 98%, el sistema completo se detendrá, llegando a haber un costo dependiente del tiempo total de la operación detenida. En cambio, dentro de los microservicios si es que "Tracking" falla, los usuarios pueden ser capaces de crear usuarios y paquetes dentro del sistema, reduciendo el impacto económico y la identificación fácil del problema.

Si se quiere desplegar una de estas arquitecturas, hay que tener en consideración el tamaño de estas. El monolito en su despliegue suele ser más demoroso, debido al tamaño del proyecto cada vez que se genera un artefacto final, teniendo que compilar todo el proyecto (ej. si se hizo un cambio de una sola línea, se compilara desde 0 el proyecto), que a diferencia de los microservicios, solo se compilará el servicio en específico, haciendo que el procesador tenga un esfuerzo menor. Por lo que, si se quisieran realizar 10 despliegues en un día, el microservicio sería más rápido generando un ahorro en tiempo y dinero.

8. Trade-offs

- **Beneficios:**
 - **Desacoplamiento:** Los fallos en el servicio de notificaciones ya no detienen la creación de paquetes
 - **Escalabilidad independiente por servicio:** Se pueden asignar más recursos al servicio de tracking durante temporadas de alto tráfico
 - **Agilidad técnica:** Es posible actualizar el servicio de auth a un lenguaje más eficiente y rápido, sin manipular el resto del sistema
- **Desventajas:**
 - **Complejidad de red:** La comunicación entre servicios introduce latencia adicional y posibles fallos de red
- **Cuándo NO usar microservicios:**
 - **Sistemas de baja complejidad** -> Si la lógica del negocio es simple y con un bajo volumen de datos, la sobrecarga de gestionar múltiples servicios supera cualquier beneficio técnico.
 - **Equipos reducidos o sin experiencia en DevOps** -> Ya que es una arquitectura distribuida, requiere de habilidades avanzadas en monitoreo/orquestación y redes.
 - **Dominios del negocio poco claros** -> Si no se comprenden los límites de cada módulo, se corre el riesgo de crear un “monolito distribuido”, en donde los servicios están tan acoplados que no pueden evolucionar de manera independiente.
 - **Necesidad de consistencia inmediata** -> Si el negocio no tolera la consistencia eventual, la complejidad de implementar transacciones de manera distribuida hace que el monolito sea una mejor opción.
 - **Presupuesto limitado de infraestructura** -> Mantener múltiples bases de datos, gateways y sistemas de notificaciones, es significativamente más caro que correr una única instancia de servidor.

9. Conclusiones

La migración de la arquitectura monolítica hacia una de microservicios permitió cumplir el objetivo principal del proyecto; desacoplar dominios críticos del servicio de tracking, mejorar la mantenibilidad del sistema y demostrar su operatividad mediante un prototipo funcional.

A lo largo del proceso, se logró migrar de un sistema centralizado a una solución distribuida con responsabilidades separadas en servicios de usuario, paquetes, tracking, incorporando API Gateway y monitoreo en tiempo real mediante Prometheus y Grafana.

Los resultados de la migración, evidencian un comportamiento coherente, ya que el monolito presenta menor costo base de infraestructura, pero baja carga sostenida con tendencia a saturarse de manera global, afectando en su latencia, throughput y CPU hasta niveles críticos. En cambio con la arquitectura de microservicios logra mantener su continuidad operativa, pero con un consumo mayor de recursos (Memoria), con cuellos de botella específicos en ciertos servicios específicos tales como el de package y tracking. Esto logra confirmar que la migración a microservicios no elimina por completo los problemas de rendimiento, pero si logra facilitar la identificación y tratamiento por cada componente, reduciendo su impacto a nivel de sistema.

Este proyecto logra demostrar que no existe una arquitectura mejor que la otra, ya que el monolito puede ser adecuado en contextos simples y de bajo crecimiento. En cambio, en microservicios promueve la escalabilidad independiente por servicio y resiliencia operativa.

10. Referencias

Shvets, A. (s. f.). *Patrones de diseño*. Refactoring.Guru.

<https://refactoring.guru/es/design-patterns>

Amazon Web Services. (s. f.). *Patrón Strangler Fig para cargas modernas*. Blog de AWS.

<https://aws.amazon.com/es/blogs/aws-spanish/patron-strangler-fig-para-cargas-modernas/>

Microsoft. (s. f.). *Circuit Breaker pattern*. Microsoft Learn.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>

IBM. (s. f.). *¿Qué es una puerta de enlace de API (API Gateway)?*.

<https://www.ibm.com/mx-es/think/topics/api-gate>

Grafana Labs. (s.f.). *Grafana Dashboards*. Recuperado el 4 de abril de 2026, de

<https://grafana.com/grafana/dashboards/>

Tigera. (s.f.). *Prometheus Grafana: Better Together*. Recuperado el 4 de abril de 2026, de

<https://www.tigera.io/learn/guides/prometheus-monitoring/prometheus-grafana/>

Miro. (s.f.). *¿Qué es un diagrama de secuencia UML?*. Recuperado el 4 de abril de 2026, de

<https://miro.com/es/diagrama/que-es-diagrama-secuencia-uml/>